

Software architecture

Software architecture

Rodrigo Martins Pagliares

Design & architecture patterns

Introduction to design and architectural patterns

Lecture: Introduction to design and architectural patterns

Module: Design & architecture patterns

Architects must understand patterns

- An architect plans software systems using patterns.
- An architect must be familiar with a variety of pattern catalogs to be effective.
- Types of patterns:
 - ✓ Architecture patterns.
 - ✓ Design patterns.

Architecture patterns

Architecture patterns define structure and behavior for systems and subsystems to support nonfunctional requirements.

Design patterns

Design patterns define structure and behavior to construct reusable software components to support functional requirements.

Enterprise architecture patterns

- **Design and architecture patterns are technology agnostic.**
 - ✓ Used, for example, by **Java, C#, Python, PHP, and Ruby** developers.
- **In Java, is common to use:**
 - ✓ **The MVC pattern** in the **Presentation layer**.
 - ✓ **Session facade pattern** in the **business layer**.
 - ✓ **DAO/Repository pattern** in the **persistence layer**.

The classical GOF design patterns

Lecture: The classical GOF design patterns

Module: Design & architecture patterns

The classical GOF design patterns

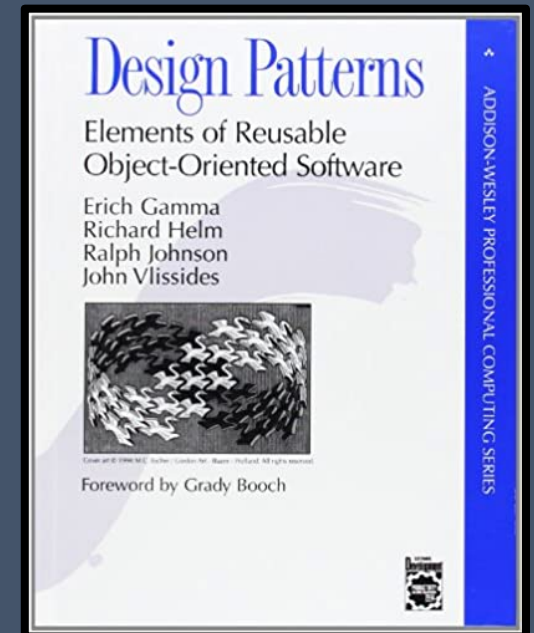
- **Design patterns** are **solutions** to **problems** already solved.
- They represent a **common vocabulary** among **developers**.
- **Example**
 - ✓ A **developer** using **one programming language** can **talk** with **developers** of **other languages** about the *singleton*, *memento*, *adapter*, etc.
 - ✓ The other **developers** will **understand** what is being **communicated**, even though they **don't know** the **language** of the **first developer**.

The classical GOF design patterns

- **Design Patterns** offer solutions to common application design problems.
- Design patterns are normally **targeted** at solving the problems associated with **object creation** and **interaction**
 - ✓ **Rather** than dealing with large scale problems faced by overall software architecture.
- **Large scale problems** are solved by enterprise patterns .

The classical GOF design patterns

- Essential tool in the programmer's toolbox ever since the Gang of Four wrote their seminal book.
- This book describes 23 design patterns divided into three categories:
 - ✓ Creational
 - ✓ Structural
 - ✓ Behavioral



The categories of the GOF design patterns

- **Creational patterns:**

- ✓ Control object creation, initialization, and class selection.

- **Behavioral patterns:**

- ✓ Control communication, messaging, and interaction between objects.

- **Structural patterns:**

- ✓ Organize relationships between classes and objects,

- ✓ provide guidelines for combining and using related objects together to achieve desired behaviors.

The 23 classical GOF design patterns

Creational

Abstract Factory
Builder
Factory method
Prototype
Singleton

Structural

Adapter
Bridge
Composite
Decorator
Facade
Flyweight
Proxy

Behavioral

Command
Interpreter
Iterator
Mediator
Memento
Chain of responsibility
Template method
Visitor
Observer
Strategy
State

The classical GOF design patterns

- Design patterns aid developing a **durable** and **maintainable** application.
 - ✓ However, they are not a **silver bullet** that **fixes** all ills.



The classical GOF design patterns

- Using design patterns unnecessarily results in a complex and overly engineered system that introduces:
 - ✓ bugs
 - ✓ inefficient design
 - ✓ low performance, and
 - ✓ maintenance issues.
- Patterns really should only be used when problems are likely to occur.

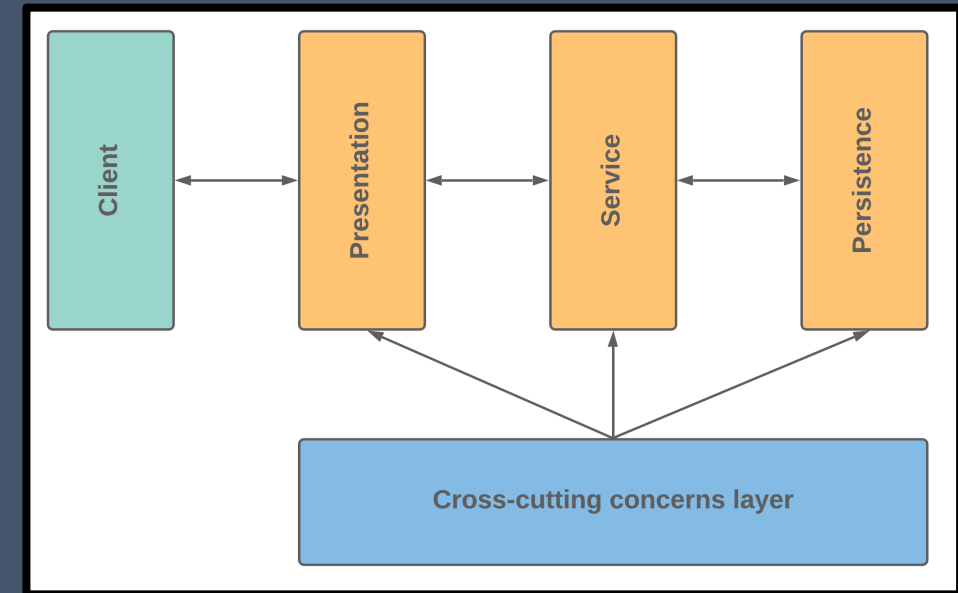
Architecture patterns

Lecture: Architecture patterns

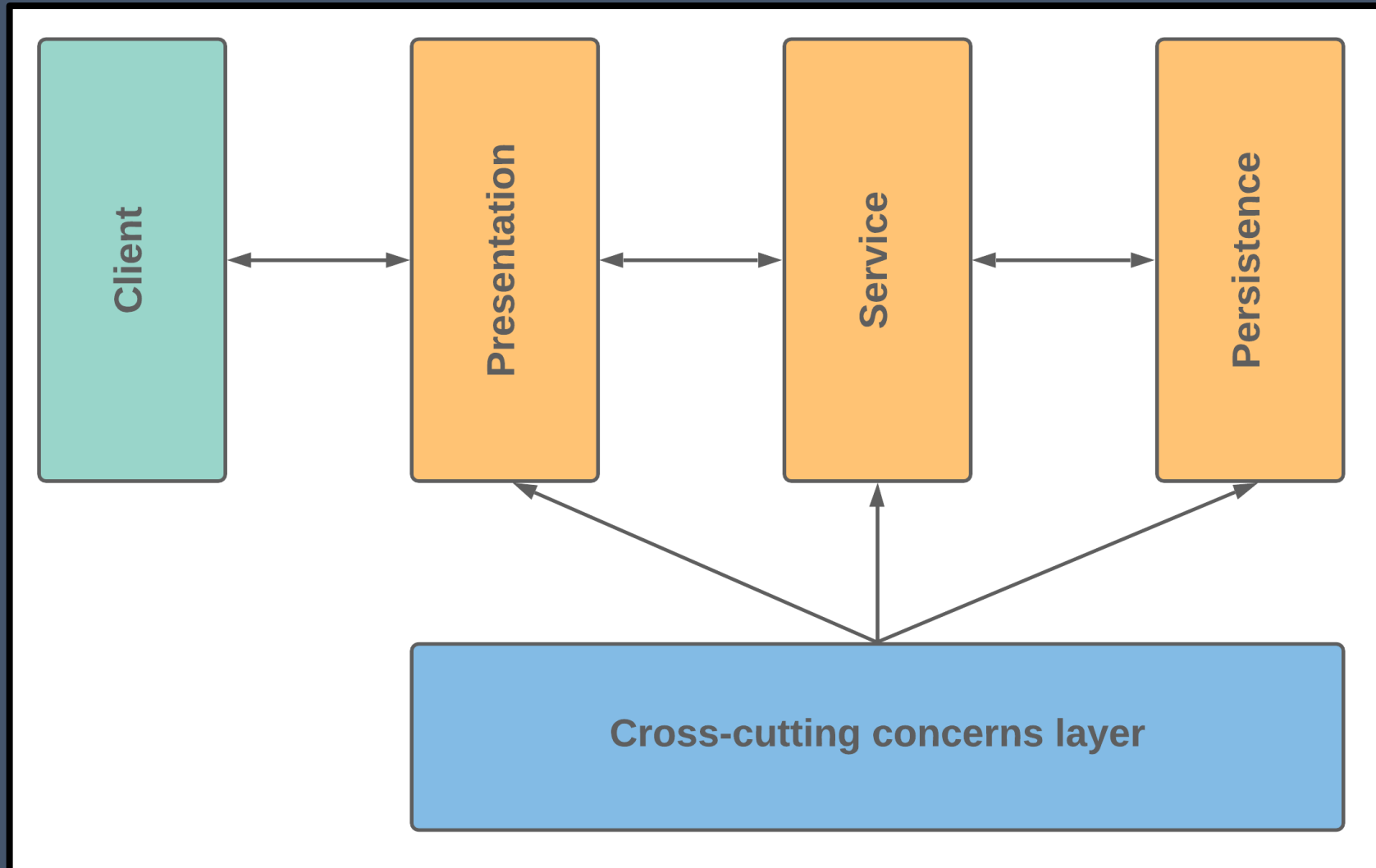
Module: Design & architecture patterns

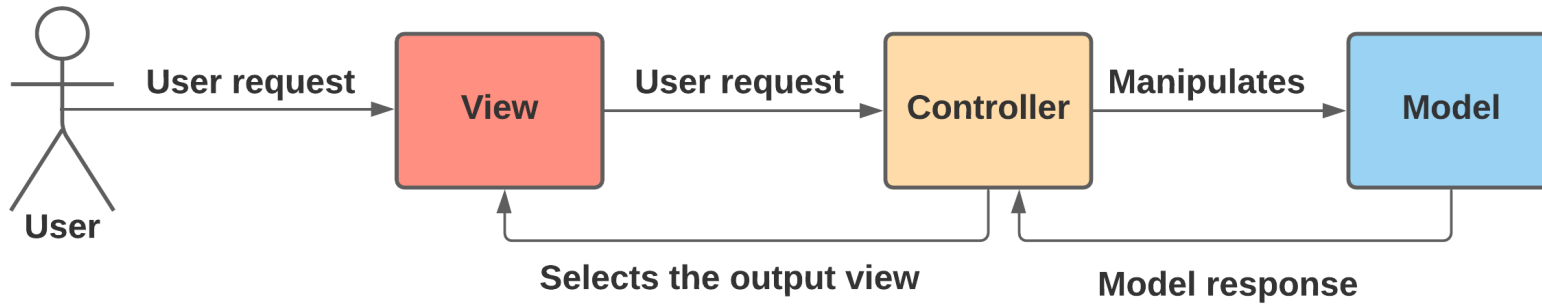
Architecture patterns

- **Architecture/enterprise patterns** resolve problems that are uniquely faced by enterprise software.
 - ✓ Deal with the big picture.
- **Architecture patterns** are usually classified by the layer of the application in which they are most commonly used.



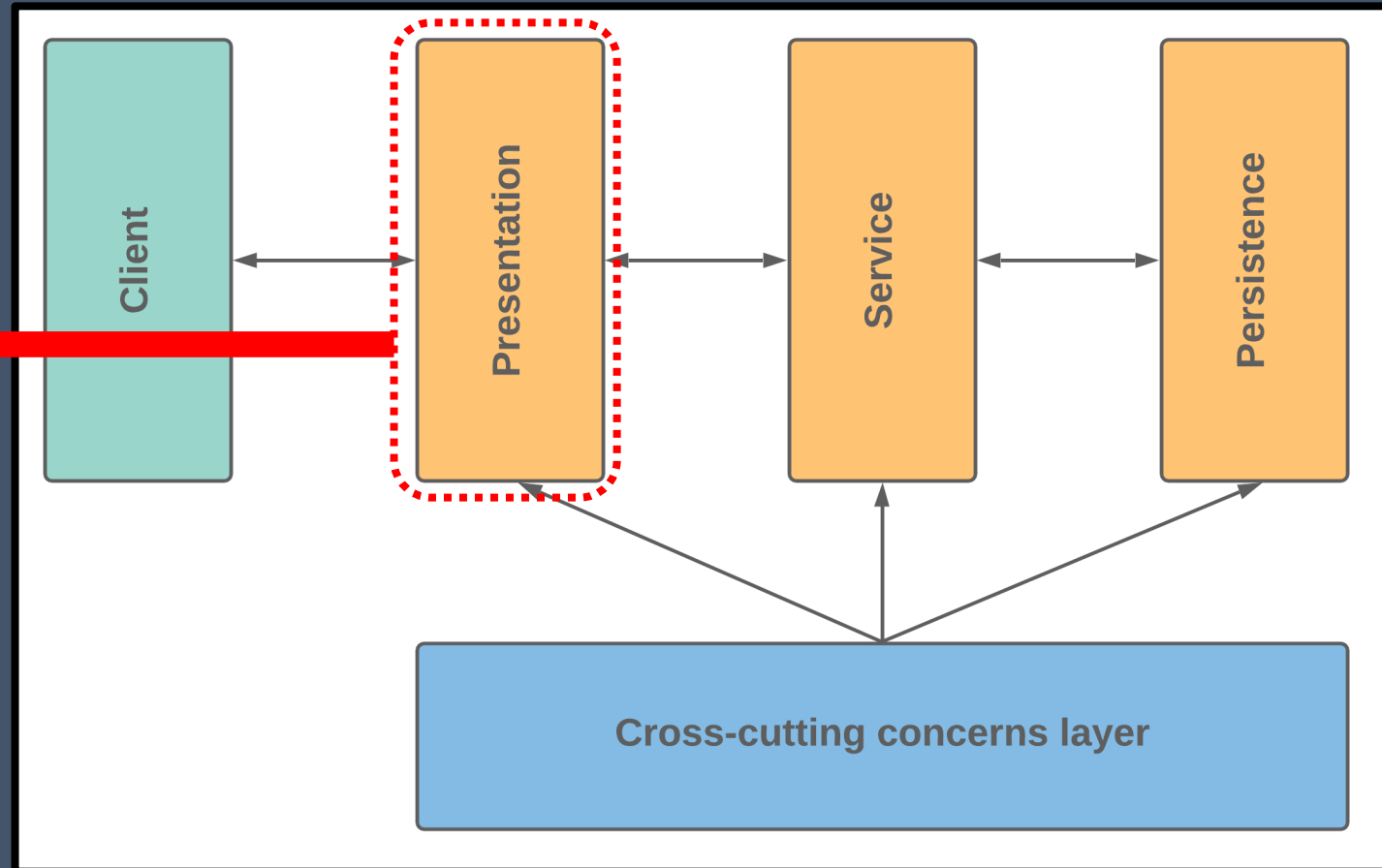
Typical layers in an enterprise architecture

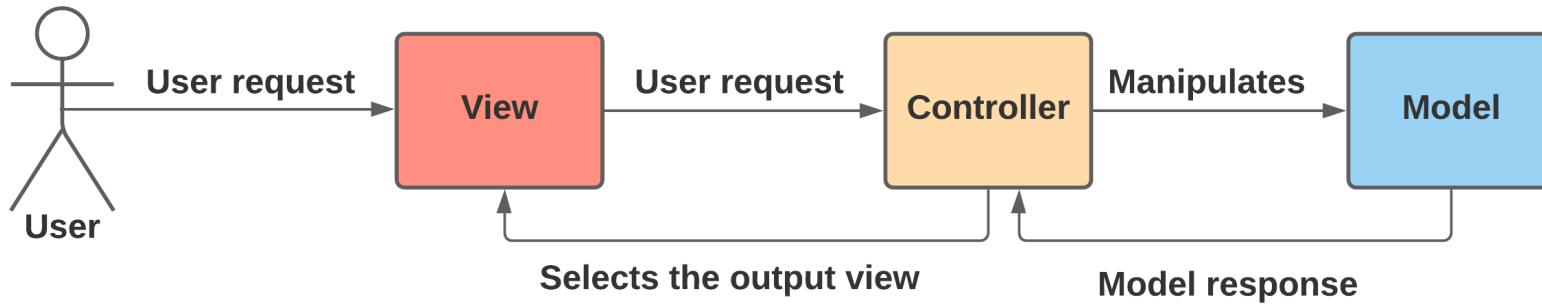




Option 1

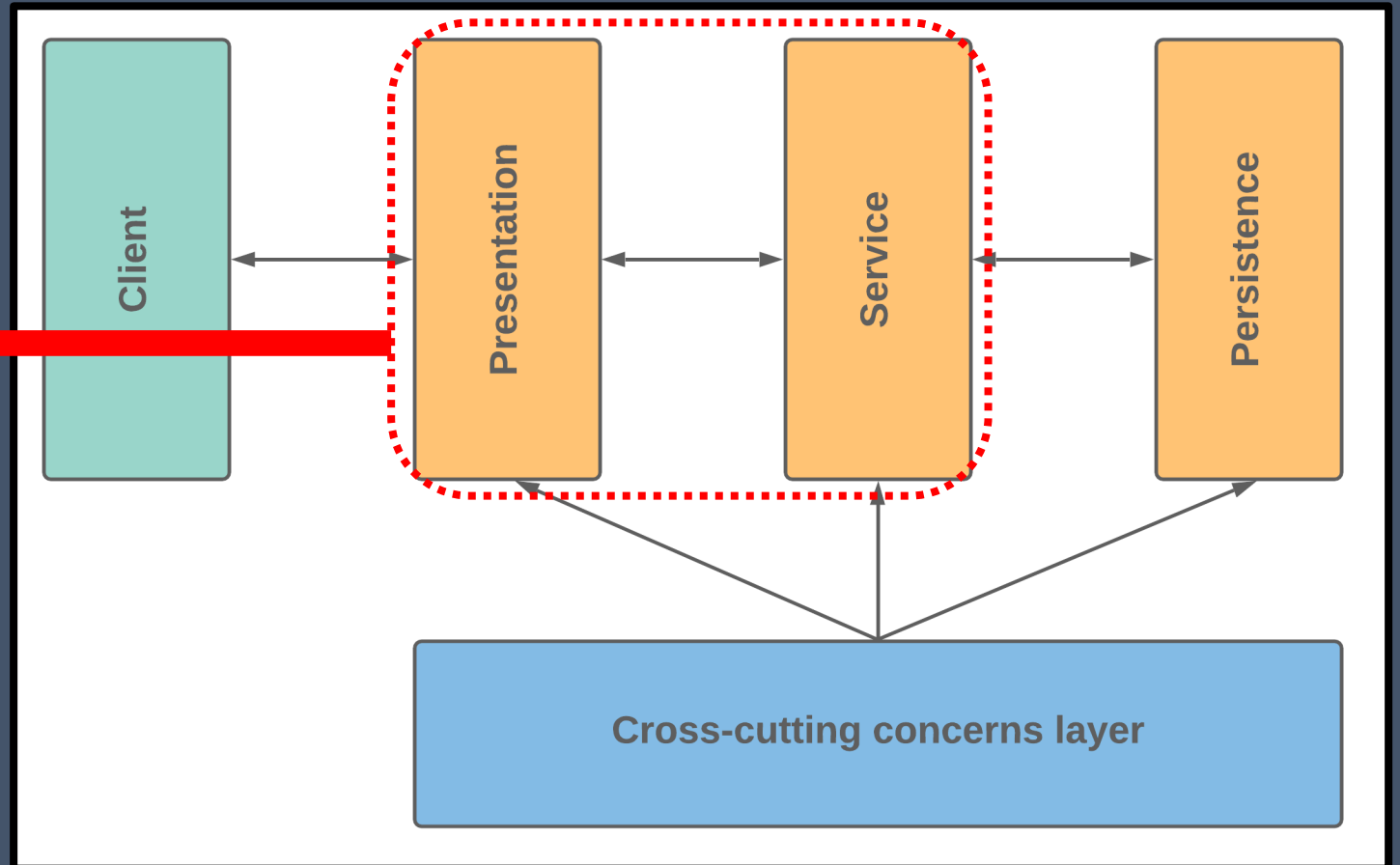
In which layer
is MVC
located?





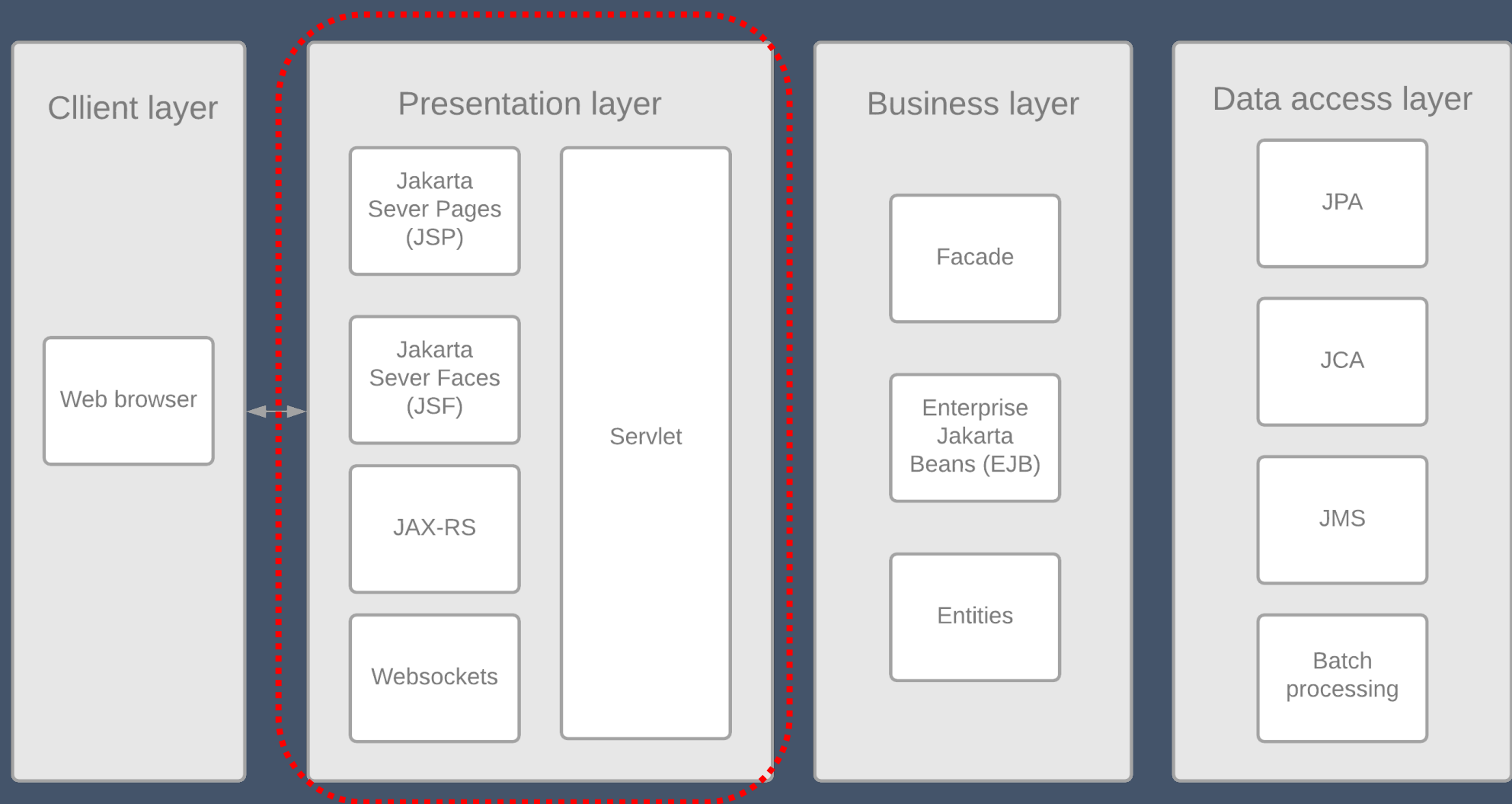
Option 2

In which layer
is MVC
located?



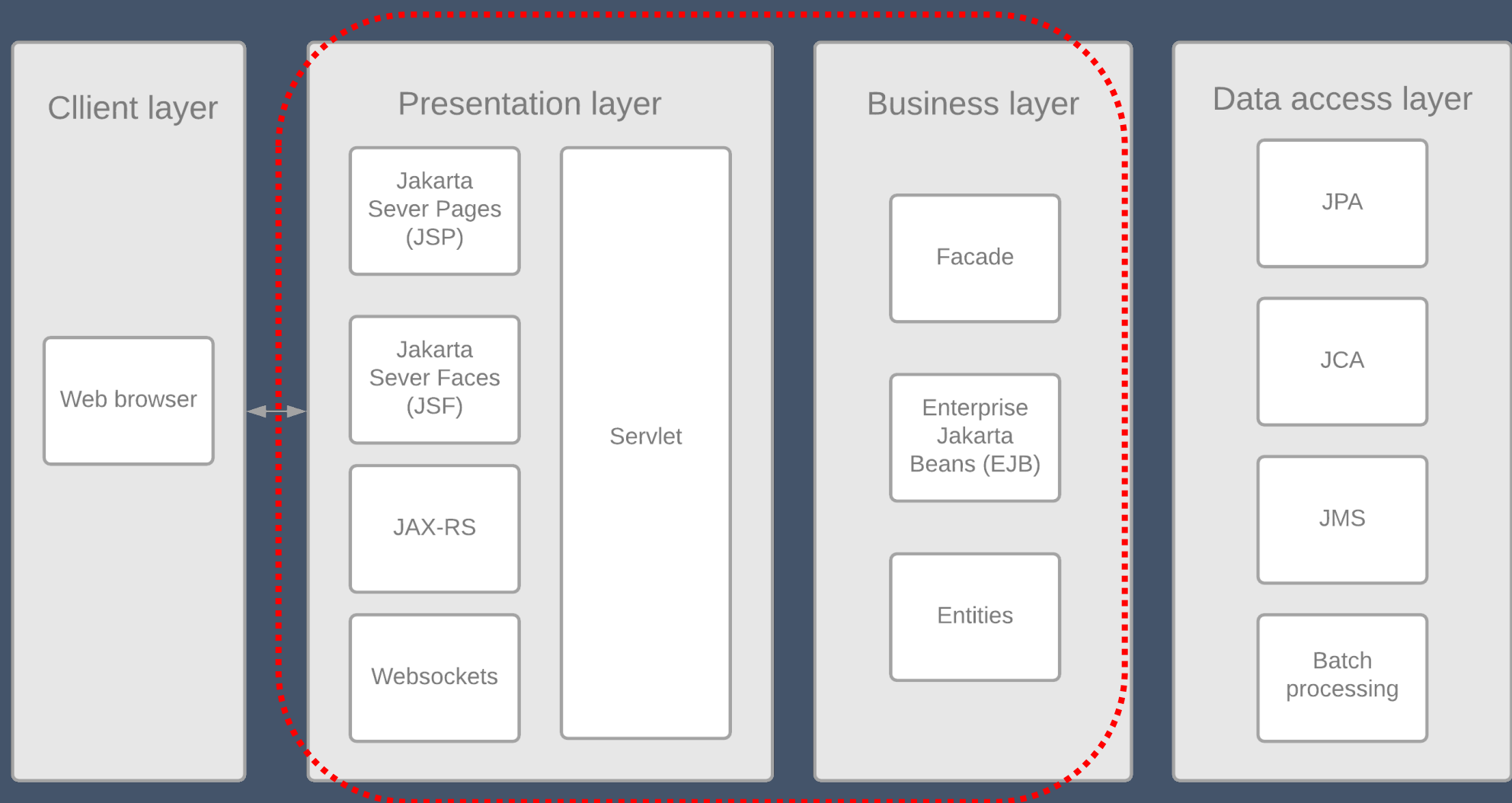
Typical architecture patterns in Jakarta EE architecture

Typical Jakarta EE architecture



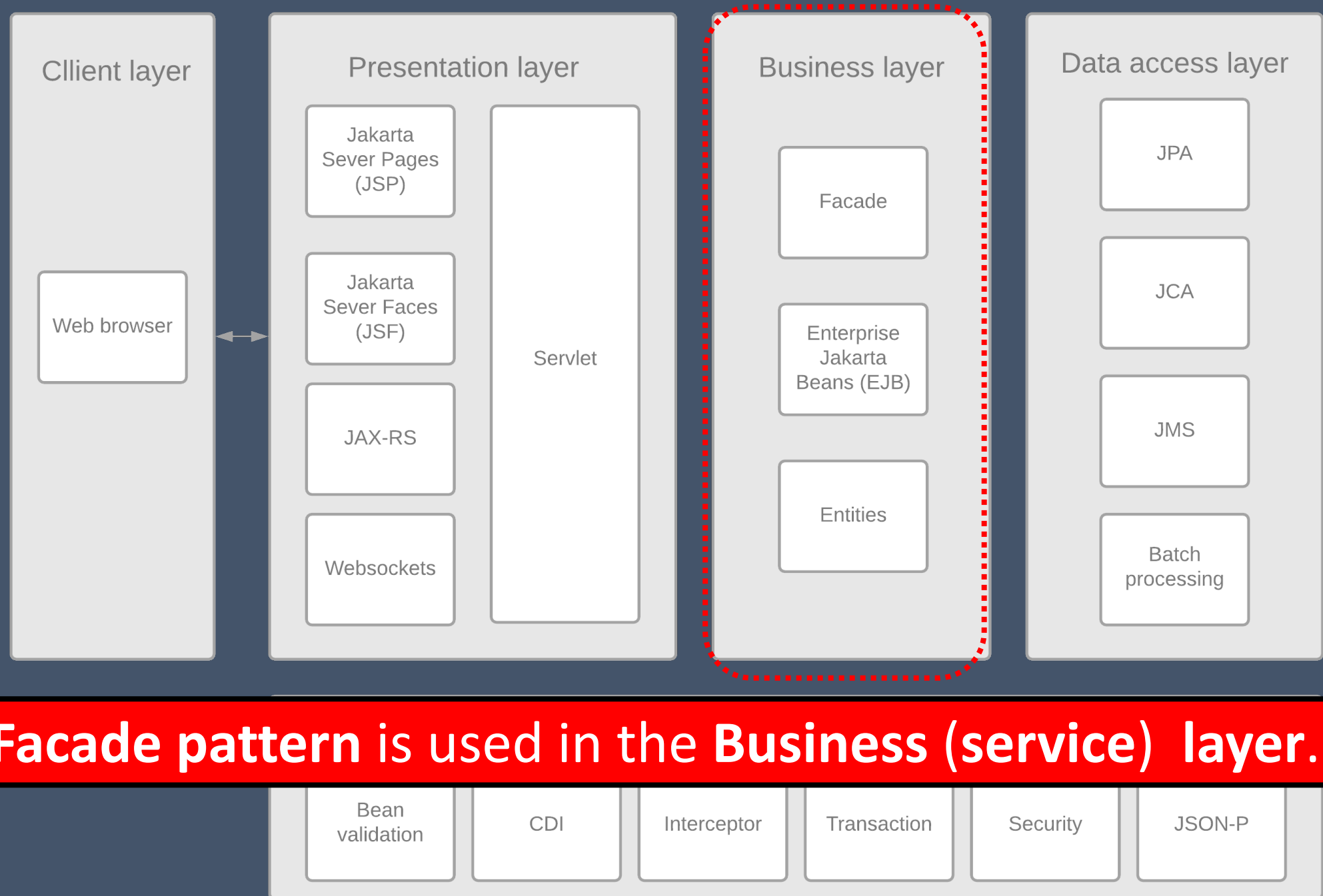
The Model-View-Controller (MVC) pattern is usually used in the Presentation layer when developing simple web applications (using servlet + JSP or by using the JSF framework, for example)

Typical Jakarta EE architecture

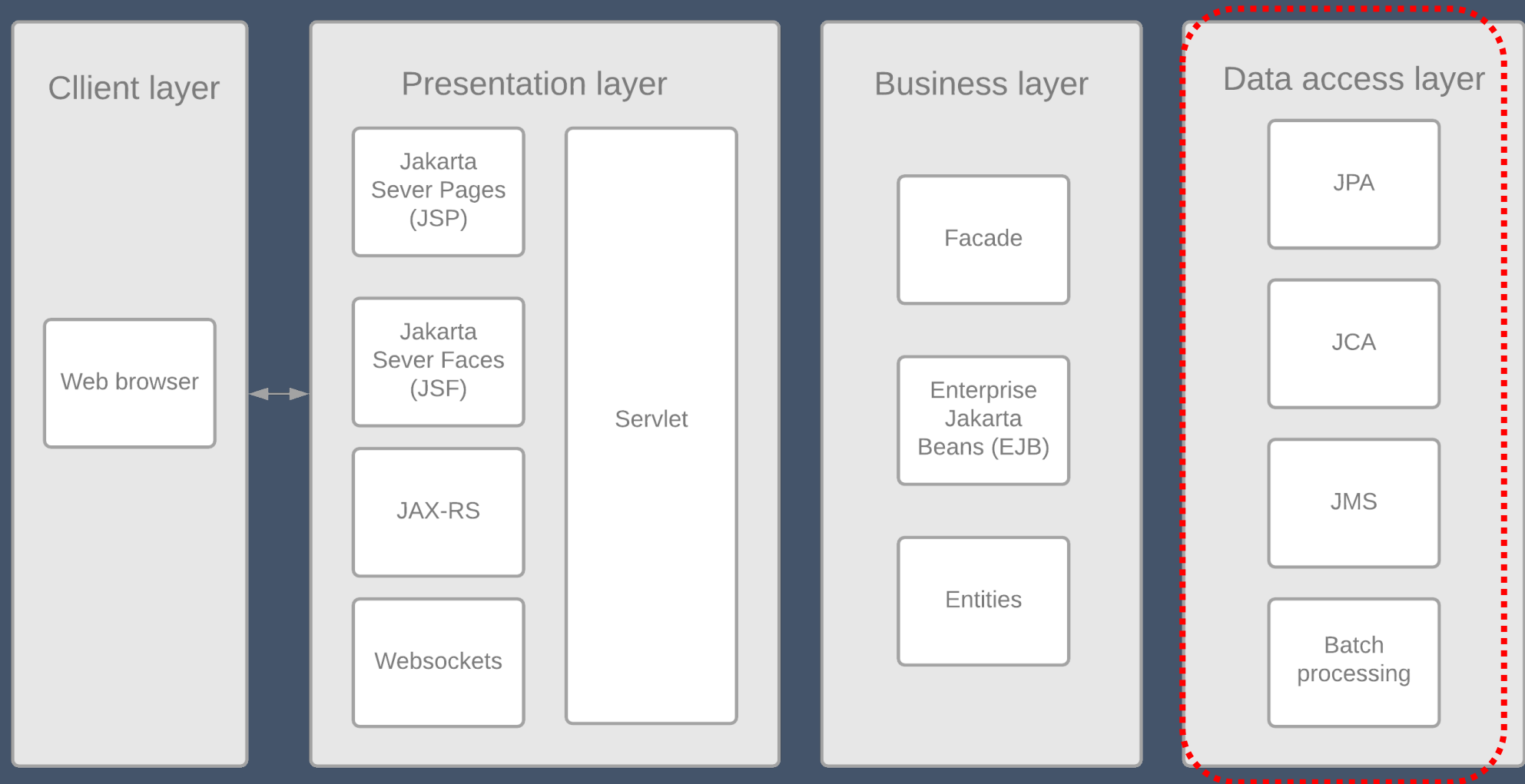


For more complex web applications, the **Model** part of the **MVC** pattern also includes elements of the **business(service)** layers (using **EJB Session beans**, for example)

Typical Jakarta EE architecture



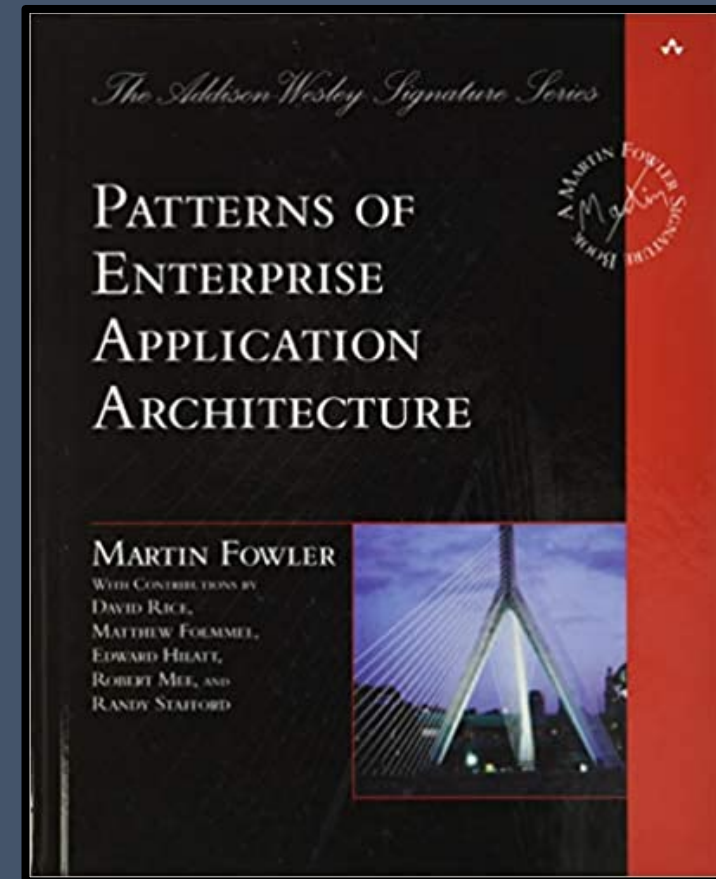
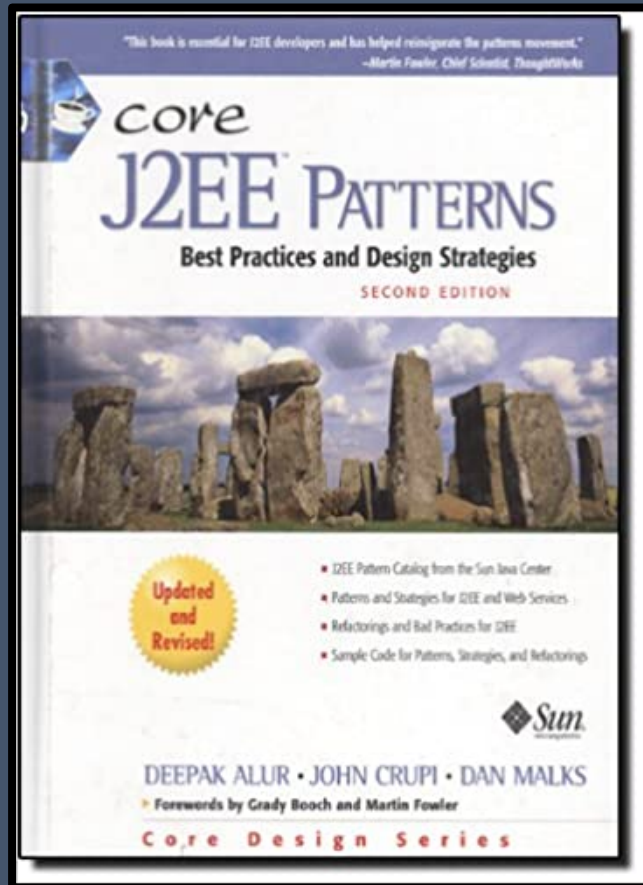
Typical Jakarta EE architecture



The Data Access Object (DAO) and Repository patterns are used in the persistence layer.

Seminal books on architecture patterns

Two seminal books on enterprise patterns

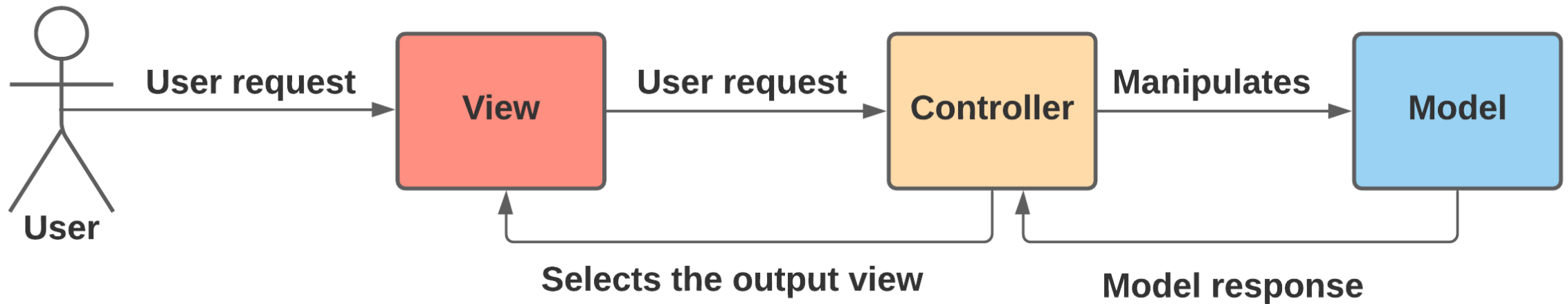


The Model-View-Controller pattern

Lecture: The Model-View-Controller pattern

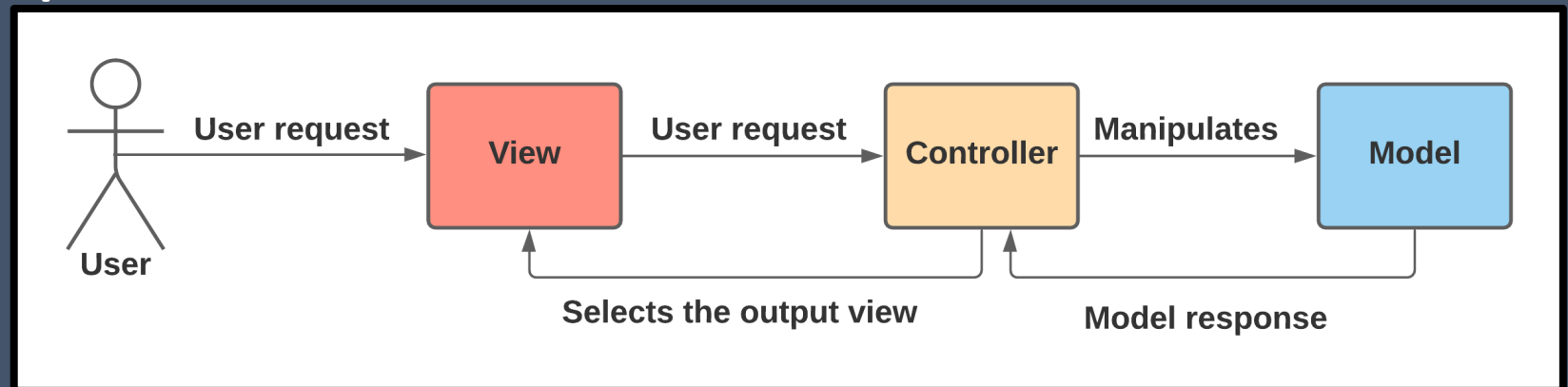
Module: Design & architecture patterns

Model-View-Controller pattern (MVC)



The MVC and separation of concerns

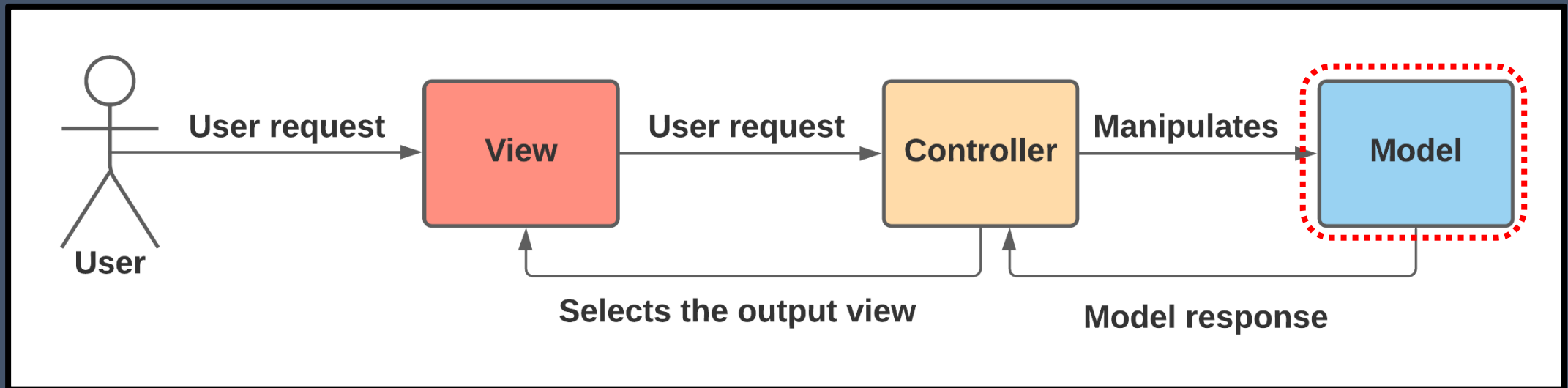
- The **Model View Controller pattern** is one of the more **ubiquitous architectural patterns** in **web application development**.
- It is **built on the philosophy of separations of concerns**.
 - **Encapsulates** and **separates the processing of application data** from the **presentation of data**.
 - **Modifications** to the **application's business logic** and **user interface** much **easier and more independent**.



Model

Model

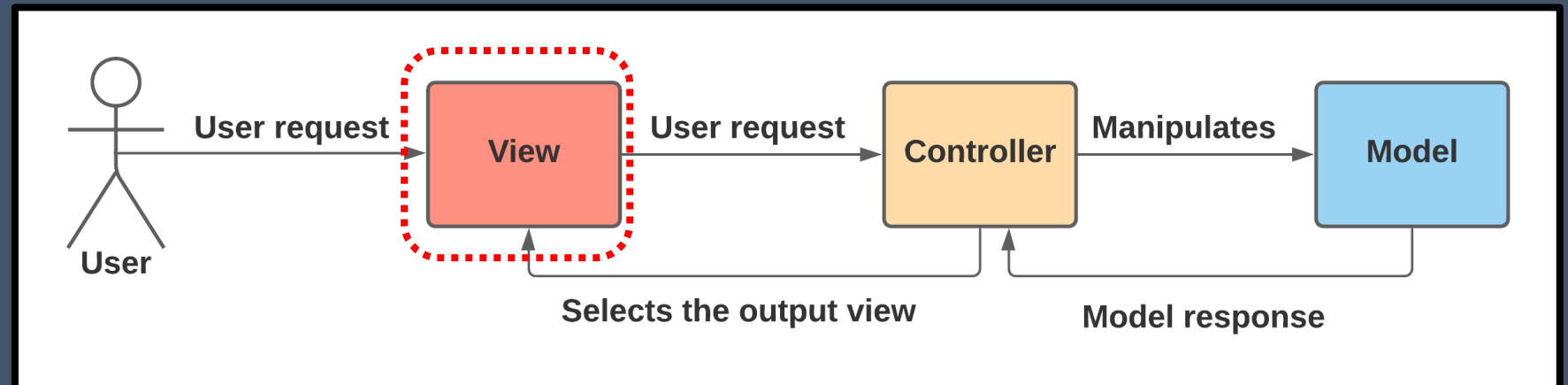
- Represents the application's data and related business logic.
- May consist of one object or a complex graph of related objects.



View

View

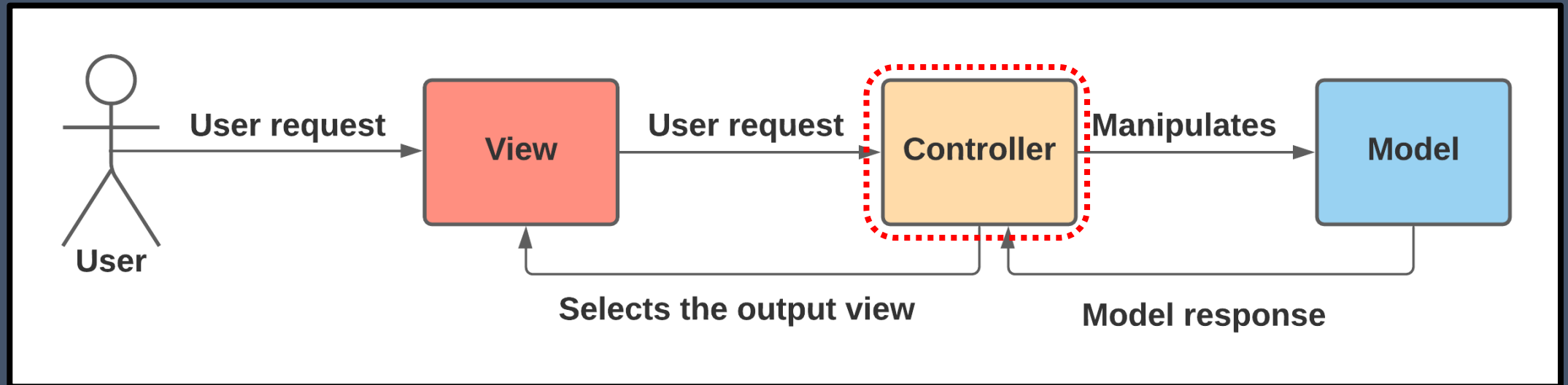
- **Visual representation** of the **data contained** in the **model**.
- A **subset** of the **model** is **represented** in a **single view**.
 - ✓ The **view** acts as a **filter** to the **model** data.
- The **user** **interacts** with the **model** **data** via the **view's** **visual** **representation**.



Controller

Controller

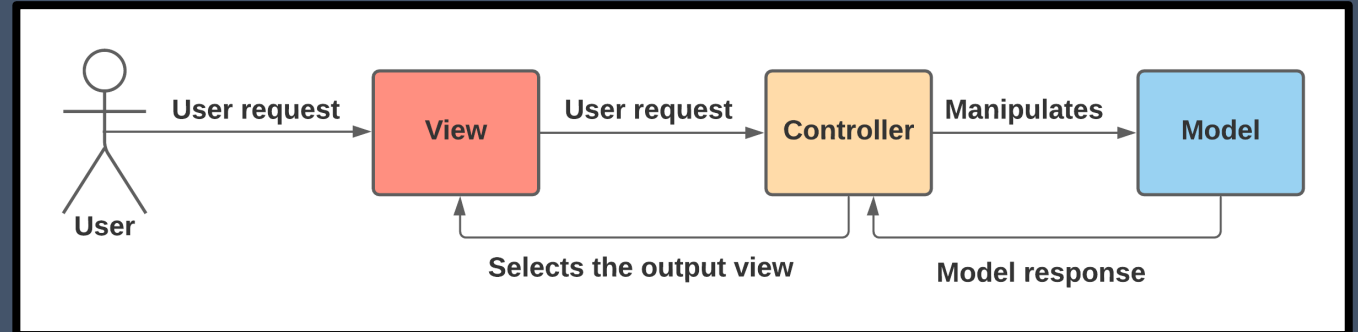
- **Links a View to the Model directing application flow.**
- **Controls which view to render to the user in response to the user's input and the business logic that it processes.**



**How do MVC components
work together?**

How do MVC components work together?

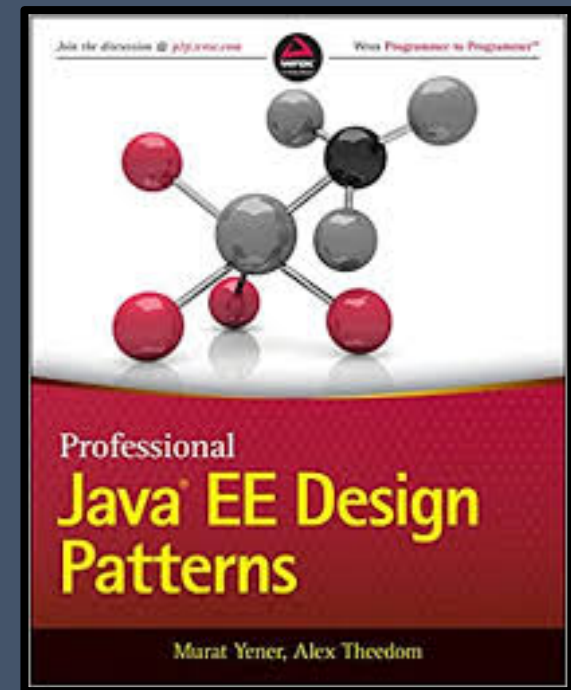
1. A **user** uses the **View** to send a message to a **Controller**.
2. The **Controller** forwards the message to the **Model**.
3. The **Model** prepares a **response** and **sends it back** to the **Controller**
4. The **Controller** choses the **View** and sent it to the **user**.
5. **Rinse and repeat !**



Recommended reading

Recommended reading

- Read Chapter 14 for more details about the MVC Pattern:
 - ✓ History
 - ✓ MVC types I and II
 - ✓ MVC in Java EE (plain code and with FacesServlet)
 - ✓ MVC in Spring
 - ✓ MVC vs. MVP (used in .NET and GWT)
 - ✓ MVC with other patterns.
 - ✓ ...



Questions?